1.Title

Receipt and Invoice Digitizer

2.Introduction

In the modern financial landscape, the management of physical receipts and invoices is often a bottleneck for both businesses and individuals. Traditional methods of handling these documents involve manual data entry, which is time-consuming, repetitive, and highly susceptible to human error. Furthermore, physical paper trails are prone to loss and deterioration, making historical tracking and audit compliance difficult.

The **Receipt and Invoice Digitizer** is a robust software solution designed to bridge the gap between physical documentation and digital accounting systems. This project aims to automate the end-to-end process of document management by transforming unstructured image data into structured, queryable financial records.

By leveraging advanced **Optical Character Recognition (OCR)** technologies (such as Tesseract or Google Vision) combined with **Natural Language Processing (NLP)** and Regex-based parsing, the system intelligently extracts critical data points—including vendor names, transaction dates, invoice IDs, and monetary values.

Beyond simple extraction, the system incorporates a layer of **logic and validation** to ensure mathematical consistency (e.g., verifying that subtotal plus tax equals the total). The processed data is securely stored in a relational **SQL database**, enabling users to generate insightful analytics, track spending trends via an interactive **Web Dashboard**, and seamlessly export data for further financial reporting.

3.Problem Statement

3.1 The Core Issue

Businesses and individuals currently rely heavily on manual workflows to manage high volumes of paper-based receipts and invoices. This dependence on physical documentation and manual data entry results in a slow, error-prone, and inefficient financial tracking process that is difficult to scale and integrate with modern digital accounting systems.

3.2 Operational Challenges

The current manual approach presents several distinct challenges that this project aims to resolve:

Inefficient Data Entry: Manually transcribing data (Vendor Name, Date, Invoice IDs, Line Items) from paper to digital spreadsheets is time-consuming and labor-intensive.

High Risk of Human Error: Manual entry is susceptible to typos, incorrect decimal placements, and calculation errors, leading to inaccurate financial reporting.

Data Organization & Retrieval: Physical receipts are easily lost, faded, or damaged over time. Searching for a specific historical transaction within a pile of paper or unstructured image folders is virtually impossible.

Lack of Validation: Without an automated system, it is difficult to instantly verify if an invoice total matches the subtotal plus tax, or to detect if a receipt has been submitted twice (duplicate entry), leading to potential financial leakage.

Unstructured Data: Raw images of receipts are "unstructured data." Without digitization, this data cannot be easily analyzed for spending trends, vendor statistics, or monthly analytics

4. Milestone 1: Document Ingestion & OCR

4.1 Description

The first milestone focuses on building the foundational pipeline required to ingest receipt documents and convert them into machine-readable text. Since receipts and invoices exist in various physical and digital formats, this stage ensures the system can accept multiple file types and prepare them for text extraction. The system provides a user-friendly interface that allows users to upload receipts in image formats such as JPG, PNG, and JPEG, as well as PDF documents. When a PDF is uploaded, the system converts it into image format so that OCR processing can be applied.

Once the document is uploaded, image preprocessing techniques are applied to improve recognition accuracy. Real-world receipts often contain noise, shadows, skew, low contrast, or background textures that interfere with OCR performance. The system processes the image

by converting it to grayscale and cleaning visual noise, thereby improving text clarity and readability.

After preprocessing, Optical Character Recognition (OCR) is applied to extract raw textual content from the receipt. The extracted text represents all visible characters from the receipt, including merchant names, dates, totals, and item descriptions. This raw text becomes the input for the next milestone, where structured data extraction occurs. This milestone is critical because the accuracy and reliability of downstream processing depend heavily on the quality of OCR output.

4.2 Objectives

4.2.1 Document Upload Interface

The system provides an intuitive upload interface that allows users to submit receipt images or PDF files directly through the web application. This interface supports drag-and-drop functionality and ensures accessibility for users with minimal technical knowledge. Supporting multiple file formats ensures the system can handle receipts captured through mobile cameras, scanners, or downloaded invoices.

4.2.2 Multi-format Document Handling

Receipts may be stored as images or PDFs. The system detects the file type and converts PDF pages into images for processing. This ensures uniformity in the OCR pipeline and allows the system to process documents regardless of their original format.

4.2.3 Image Preprocessing for OCR Accuracy

Before text extraction, the uploaded image undergoes preprocessing to improve OCR performance. Converting the image to grayscale reduces noise and enhances contrast between text and background. This improves character recognition accuracy and reduces OCR errors caused by shadows, colors, or textured backgrounds.

4.3.4 OCR-based Text Extraction

The system uses Optical Character Recognition to convert the processed image into raw text. OCR scans the image pixel by pixel and identifies alphanumeric characters and symbols. This text includes vendor details, dates, totals, and item names, forming the basis for structured data extraction in later stages.

4.4 Outcomes

By the end of this milestone, the system successfully accepts receipt documents in multiple formats and converts them into clean, machine-readable text. Users can upload receipts

through an easy-to-use interface, and the system preprocesses images to improve text clarity. OCR extraction produces raw textual data that accurately reflects the content of the receipt. This milestone establishes the foundation for structured data extraction by ensuring reliable text input for subsequent processing stages.

5. Milestone 2: Field Extraction & Validation

5.1 Description

The second milestone focuses on transforming unstructured OCR text into structured financial data. Raw OCR output contains all visible text but lacks organization and meaning. This stage applies AI-based parsing, natural language processing techniques, and rule-based extraction methods to identify and extract important receipt fields such as merchant name, date, invoice number, subtotal, tax, total amount, category, and line items.

The system uses AI parsing via a language model to interpret receipt text and convert it into structured JSON data. This approach improves accuracy when receipts have inconsistent layouts. After extraction, validation logic is applied to ensure data correctness and integrity. The system verifies that subtotal plus tax equals the total, identifies suspicious tax rates, and checks for missing required fields.

Duplicate detection is another critical function implemented in this milestone. The system compares vendor name, date, total amount, and invoice number to detect repeated receipts. This prevents duplicate financial entries and maintains database integrity.

5.2 Objectives

5.2.1 Structured Field Extraction

The system analyzes OCR text and extracts meaningful financial fields such as merchant name, transaction date, subtotal, tax, and total amount. AI-assisted parsing enables accurate extraction even when receipt layouts vary.

5.2.2 Line-Item Extraction

In addition to summary fields, the system extracts individual line items, including product names, quantities, and prices. This enables detailed spending analysis and item-level insights.

5.2.3 Data Validation and Integrity Checks

The system verifies that extracted totals are mathematically correct by checking whether subtotal plus tax equals the total amount. It also flags abnormal tax rates and ensures required fields are present.

5.2.4 Duplicate Receipt Detection

To maintain data accuracy, the system checks whether a similar receipt already exists. Duplicate detection is performed using invoice numbers, dates, vendor names, and total amounts.

5.2.5 Data Categorization

The system assigns categories such as groceries, dining, shopping, or transportation to receipts. Categorization enables spending analysis and budgeting insights.

5.4 Outcomes

At the completion of this milestone, the system converts raw OCR text into structured, validated financial records. The extracted data includes vendor details, transaction date, totals, and itemized purchases. Validation logic ensures accuracy and flags inconsistencies, while duplicate detection prevents redundant entries. This milestone transforms raw text into meaningful financial data ready for storage and analysis.

6. Milestone 3: Dashboard & Reporting

6.1 Introduction

This milestone focuses on developing an interactive dashboard that allows users to review, manage, and export digitized receipts. The dashboard provides a centralized interface where users can view uploaded receipts, verify extracted data, and access financial summaries. A well-designed dashboard improves usability and ensures users can interact with the system efficiently.

The system enables users to browse receipt history, view itemized details, and delete records if necessary. Export functionality allows users to download receipt summaries and item-level data in CSV or Excel format, enabling integration with accounting tools.

Additionally, this milestone introduces basic analytics such as monthly spending totals and vendor summaries. These analytics provide insights into spending behavior and financial trends.

6.2 Objectives

6.2.1 Interactive Receipt Management Dashboard

The system provides a visual interface for viewing, managing, and verifying receipts. Users can browse uploaded receipts and inspect extracted details.

6.2.2 Receipt History & Detail View

Users can view complete receipt information including vendor name, date, totals, and itemized purchases.

6.3.3 Data Export Functionality

The system allows users to export receipt data and line items in CSV and Excel formats for accounting or reporting purposes.

6.3.4 Basic Spending Analytics

The dashboard displays summary statistics such as monthly spending totals and vendor-wise expenditure, providing users with quick financial insights.

6.5 Outcomes

By the end of this milestone, users can interact with their digitized receipts through an intuitive dashboard. They can review receipt details, manage records, and export data for external use. Basic analytics provide insights into monthly spending and vendor patterns, improving financial awareness and decision-making.

7. Milestone 4: Polishing & Integration

7.1 Introduction

The final milestone enhances system accuracy, performance, and usability. Template-based parsing is introduced to improve extraction accuracy for frequently used vendors by applying known tax rates and standardized naming conventions. This significantly reduces errors and improves consistency.

Advanced search and filtering capabilities are added to help users quickly locate receipts based on vendor, category, date range, or amount. Database queries are optimized, and caching mechanisms are implemented to improve system performance and responsiveness.

This milestone also introduces budgeting features and advanced analytics, enabling users to monitor spending patterns and receive AI-generated budget recommendations.

7.2 Objectives

7.2.1 Template-Based Parsing for Accuracy Improvement

The system uses vendor templates to standardize merchant names and apply known tax rates. This improves extraction accuracy and ensures consistent financial records.

7.2.2 Advanced Search and Filtering

Users can search receipts using keywords and apply filters such as date range, category, merchant, and amount. This improves accessibility and usability.

7.2.3 Database Optimization & Performance Enhancement

Indexes and caching are implemented to speed up queries and reduce system load, ensuring smooth performance even with large datasets.

7.2.4 Budget Tracking & AI Recommendations

The system allows users to set monthly budgets and receive AI-generated suggestions when spending exceeds limits.

7.2.5 Enhanced Analytics & Insights

Advanced analytics provide trends, category spending breakdowns, and behavioral insights to support financial decision-making.

7.4 Outcomes

At the completion of this milestone, the system delivers a polished, intelligent financial management solution. Template-based parsing improves accuracy, while advanced search and optimized queries enhance usability and performance. Budget tracking and analytics empower users with insights into their spending habits, enabling better financial planning and decision-making.

8. Database Schema Design

8.1 Overview

The database schema for the Receipt & Invoice Digitizer is designed to store structured receipt data, itemized purchases, and budgeting information in an efficient and searchable format. The system uses a relational database (SQLite) to maintain data integrity, enable fast querying, and support analytics and reporting.

The schema is normalized to avoid redundancy and to allow efficient relationships between receipts and their associated line items. Each receipt acts as a parent record containing summary information, while individual products purchased are stored as child records linked through foreign keys. Additionally, a budgeting table is included to support monthly spending tracking and financial planning.

This database design supports search functionality, duplicate detection, analytics generation, and dashboard reporting.

Receipts Table

Column Name	Data Type	Description
id	INTEGER (PK)	Unique identifier for each receipt
merchant	TEXT	Name of the vendor/store
date	TEXT	Transaction date (YYYY-MM-DD)
invoice_number	TEXT	Invoice or receipt number
subtotal	REAL	Total before tax
tax	REAL	Tax amount
total_amount	REAL	Final total amount paid
filename	TEXT	Original uploaded file name
upload_timestamp	TEXT	Date & time when uploaded
category	TEXT	Expense category (Groceries, Dining, etc.)

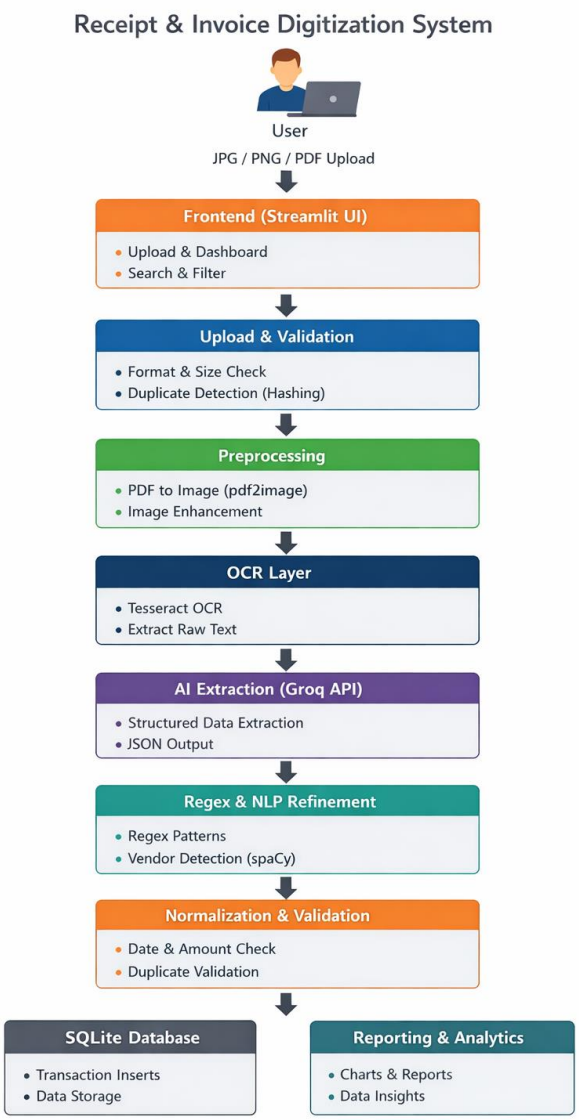
Line Items Table

Column Name	Data Type	Description
id	INTEGER (PK)	Unique identifier for each line item
receipt_id	INTEGER (FK)	Links item to a receipt
name	TEXT	Item/product name
qty	INTEGER	Quantity purchased
price	REAL	Price per unit

Monthly Budgets Table

Column Name	Data Type	Description
id	INTEGER (PK)	Unique identifier
month_year	TEXT (UNIQUE)	Month in YYYY-MM format
budget_amount	REAL	Budget limit for the month

9. System Architecture



10. Technology Stack

Layer	Technology	Detail
Frontend Layer	Streamlit	Used for building interactive web interface and dashboard for receipt management.
Frontend Layer	Plotly	Used for generating dynamic charts and visual analytics reports.
Backend Layer	Python	Core programming language used for

		implementing business logic and data processing.
Backend Layer	SQLite	Lightweight relational database used for storing invoice and receipt records.
Backend Layer	JSON	Used for structured data exchange between AI model and application.
OCR Layer	Tesseract OCR	Used to extract raw text from scanned receipts and invoice images.
OCR Layer	pdf2image	Converts PDF documents into images for OCR processing.
AI Extraction Layer	Gemini AI	Used for structured data extraction and JSON-formatted output generation.
AI Extraction Layer	Regex (re module)	Used for pattern-based extraction of dates, totals, tax amounts, and currency fields.
NLP Layer	spaCy	Used for contextual vendor name detection and entity recognition.
Data Validation Layer	Hashlib	Used for file hashing to detect duplicate uploads.
Data Validation Layer	Custom Validation Logic	Ensures format validation, file size control, and schema enforcement.
Database Safety Layer	Transaction Management	Ensures safe inserts with rollback on failure.
Exception Handling Layer	Python Exception Handling	Structured try-except blocks to prevent system crashes.
Reporting Layer	Pandas	Used for data manipulation and

		generating structured reports.
Deployment Layer	Local Host / Web Deployment	Application runs locally or can be deployed on cloud platforms.

11. Modules Implemented

1. Document Ingestion

This is the entry point of the system, responsible for accepting files and preparing them for processing.

File Upload: Users can upload receipts and invoices in various formats, such as PDFs or standard image files (JPG, PNG).

Preprocessing: Before text can be read, the image is cleaned up. This involves resizing, denoising (removing graininess), and binarization (converting to black and white) to ensure the OCR engine gets the clearest possible input.

2. OCR & Text Extraction

Once the image is clean, this module focuses on turning the visual data into machine-readable text.

Core Technology: It utilizes **Tesseract OCR** or the **Google Vision API** to scan the document.

Layout Handling: It is designed to handle complex formatting, such as skewed scans (crooked images), multi-column layouts, or tabular data often found on invoices.

Output: The result is "raw text"—a continuous stream of characters extracted from the image.

3. Field Extraction & Validation

This is the "brain" of the project. It takes the raw text from the previous step and makes sense of it.

Parsing: Using **NLP (Natural Language Processing)** and **Regex (Regular Expressions)**, the system identifies specific data points:

Dates, Vendor Names, Invoice IDs

Tax amounts, Line items, and Total amounts

Validation Logic: It performs math checks (e.g., ensuring Subtotal + Tax = Total) to verify accuracy.

Duplicate Detection: It checks invoice numbers or generates hashes to ensure the same receipt isn't processed twice.

4. Database Storage

After the data is cleaned and validated, it needs a permanent home.

Structure: The extracted data is stored in a structured **SQL database**.

Searchability: This allows users to query the data later, such as searching for all receipts from a specific "Vendor" or finding expenses within a certain date range or amount.

5. Dashboard & Reporting

This is the user interface (UI) module where the user interacts with the system.

Web Dashboard: Built using **Streamlit** or **Flask**, this interface allows users to upload files and manually review the extracted data.

Exporting: Users can download their data in **CSV** or **Excel** formats for use in other software.

Analytics: The dashboard provides visual summaries, such as monthly spending trends or vendor-specific statistics.

`6. field_extractor.py`

Purpose: Deterministic Field Extraction (Regex + AI Fallback Support)

Responsibilities:

- Extract invoice number using regex
- Extract dates (multiple formats)
- Extract subtotal, tax, total
- Extract currency symbols / ISO codes
- Extract line items (basic pattern detection)
- Recover weak or missing AI fields

`7. vendor_identifier.py`

Purpose: NLP-Based Vendor Name Identification

Responsibilities:

- Analyze OCR header text
- Use positional importance (top lines weighted higher)
- Use capitalization heuristics
- Remove keywords like INVOICE, TAX, BILL, RECEIPT
- Detect legal suffixes (LLC, LTD, PVT LTD, INC)
- Use spaCy NER for ORGANIZATION entity detection

8. `normalization.py`

Purpose: Structured Data Normalization & Standardization

Responsibilities:

- Convert all text to uppercase
- Convert date formats → ISO 8601 (`YYYY-MM-DD`)
- Enforce numeric type safety
- Trim long strings
- Apply default values for missing fields
- Format currency fields safely

9. `currency_converter.py`

Purpose: Multi-Currency Handling & USD Standardization

Responsibilities:

- Detect currency from symbol/code
- Convert all amounts → USD
- Store:
  - Original amount
  - Original currency
  - Exchange rate used
  - USD converted value
- Preserve audit transparency

10. `amount_validator.py`

Purpose: Robust Amount Validation Engine

Responsibilities:

- Validate:
  - Subtotal + Tax = Total
- Support:
  - Tax-inclusive models
  - Tax-exclusive models
- Allow tolerance for OCR rounding errors
- Generate warnings instead of blocking
- Detect financial inconsistencies

Output:

- `is_valid`
- `warning_messages`
- `validation_confidence`

11. `duplicate_detector.py`

Purpose: Logical Duplicate Detection

Responsibilities:

- Generate business-key hash:
  - Vendor
  - Invoice number
  - Date
  - Total
- Compare against DB entries
- Detect duplicates across different file uploads
- Prevent double storage
- No schema modification required

Key Technologies:

- SHA256 hashing
- SQLite lookup queries

12. `confidence_scorer.py`

Purpose: OCR & AI Confidence Scoring

Responsibilities:

- Evaluate:
  - OCR confidence %
  - Missing fields
  - AI output completeness
- Assign:
  - High / Medium / Low confidence
- Flag low confidence for review
- Trigger fallback extraction if needed

13. `validation_controller.py`

Purpose: Central Validation & Feedback Manager

Responsibilities:

- Coordinate:
  - Amount validation
  - Duplicate detection
  - Missing field checks
- Collect warnings
- Pass feedback to UI
- Ensure non-disruptive user experience

`database_manager.py`

Purpose: Persistent Relational Storage (SQLite)

Responsibilities:

- Transaction-safe inserts
- Rollback on failure
- Maintain foreign key integrity
- Store:
  - Bills table
  - Line items table
  - Audit raw OCR JSON
- Indexed querying for:
  - Vendor
  - Date
  - Amount
  - Payment method

14. `audit_logger.py`

Purpose: Audit Trail & Error Logging

Responsibilities:

- Store raw OCR output

- Log normalization changes
- Log validation warnings
- Store fallback corrections
- Track retry attempts
- Maintain error logs without crashing system

15. `dashboard_app.py`

Purpose: Main Streamlit Application Controller

Responsibilities:

- Initialize Streamlit app layout
- Configure sidebar navigation
- Route between:
    - Upload & Review
    - History Page
    - Analytics Page
- Load shared database connection
- Maintain session state

16. `upload_review_module.py`

Purpose: Upload & Extracted Data Review Interface

Responsibilities:

- Handle image/PDF upload
- Display original receipt image
- Trigger extraction pipeline
- Show extracted fields:
    - Vendor
    - Date
    - Invoice number
    - Subtotal
    - Tax
    - Total
    - Category
- Display line items in tabular format
- Show validation warnings (if any)
- Allow user confirmation before save

17. `export_manager.py`

Purpose: CSV & Excel Export Engine

Responsibilities:

- Export receipt-level summary (1 row per receipt)
- Export line-item-level data
- Generate:
  - CSV file
  - Excel file (.xlsx)
- Ensure formatted numeric columns
- Maintain timestamped filenames

18. `ai_budget_suggestions.py`

Purpose: AI-Based Budget Optimization

Responsibilities:

- Trigger when:
  - Monthly spending > Budget
- Prepare summary prompt:
  - Total spent
  - Category breakdown
- Send request to AI API
- Display practical suggestions
- Non-mandatory feature (optional display)

19. `template_engine.py`

Purpose: Deterministic Vendor Template System

Responsibilities:

- Maintain `VENDOR_TEMPLATES` dictionary
- Map vendor aliases → standardized name
- Store vendor metadata:
  - Tax rate
  - Category
  - Known formatting patterns

- Apply deterministic correction after AI extraction

20. `advanced_filter_engine.py`

Purpose: Dynamic SQL Query Builder

Responsibilities:

- Accept filter parameters:
  - Text search
  - Date range
  - Category
  - Amount range
- Build parameterized SQL query
- Prevent SQL injection
- Return filtered dataset

21. `index_manager.py`

Purpose: SQLite Index Creation & Maintenance

Responsibilities:

- Create B-Tree indexes on:
  - receipt_date
  - vendor
  - category
  - invoice_number
- Ensure indexes exist at startup

22. `query_optimizer.py`

Purpose: Server-Side Aggregation Engine

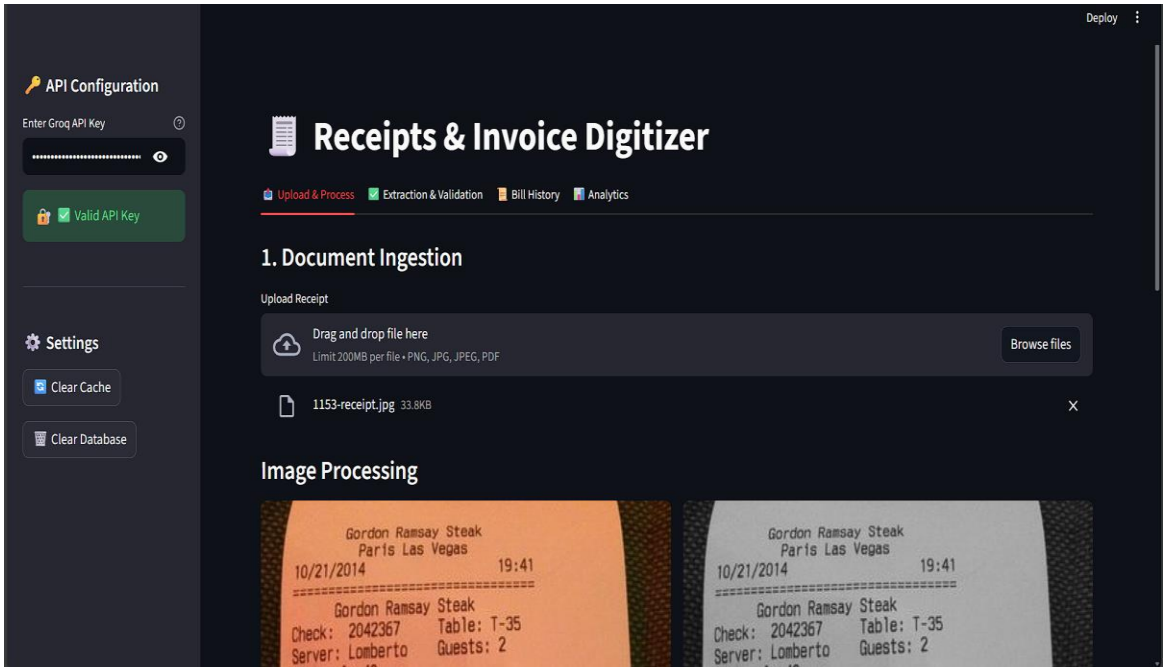
Responsibilities:

- Replace Python aggregation with SQL:
  - SUM()
  - COUNT()
  - AVG()

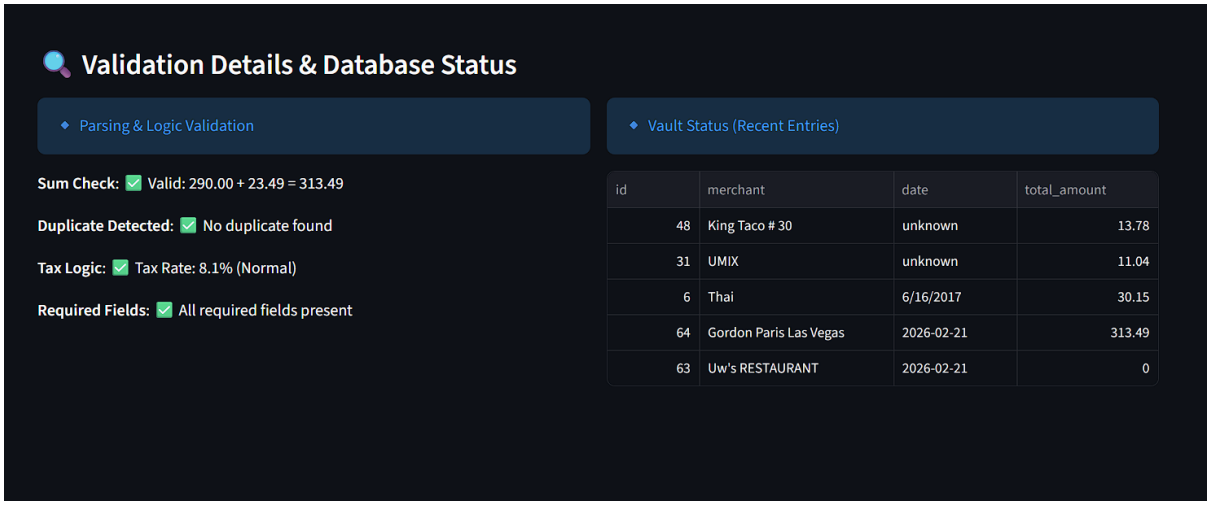
    - GROUP BY
- Pre-aggregate:
    - Monthly totals
    - Vendor totals
    - Category spending
- Reduce memory load

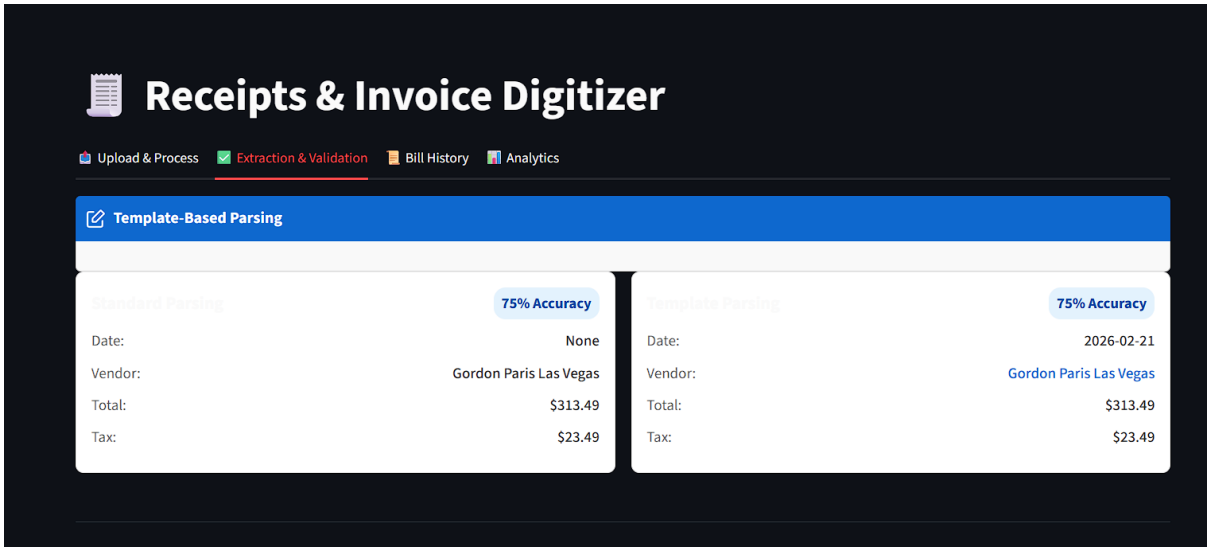
12. Result Display

1 Input

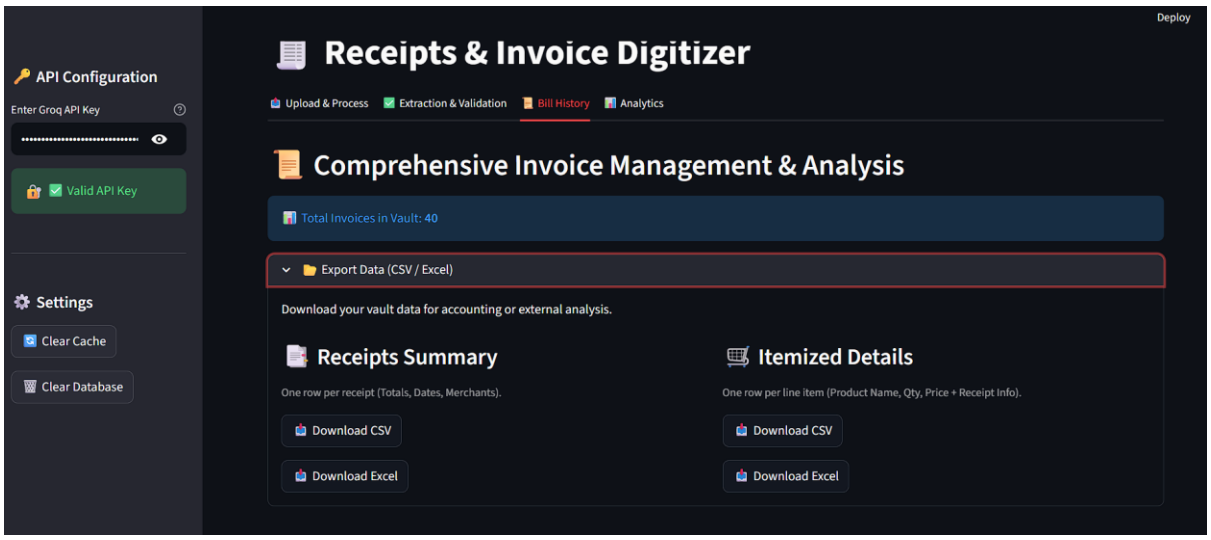


2 Validation And Extraction

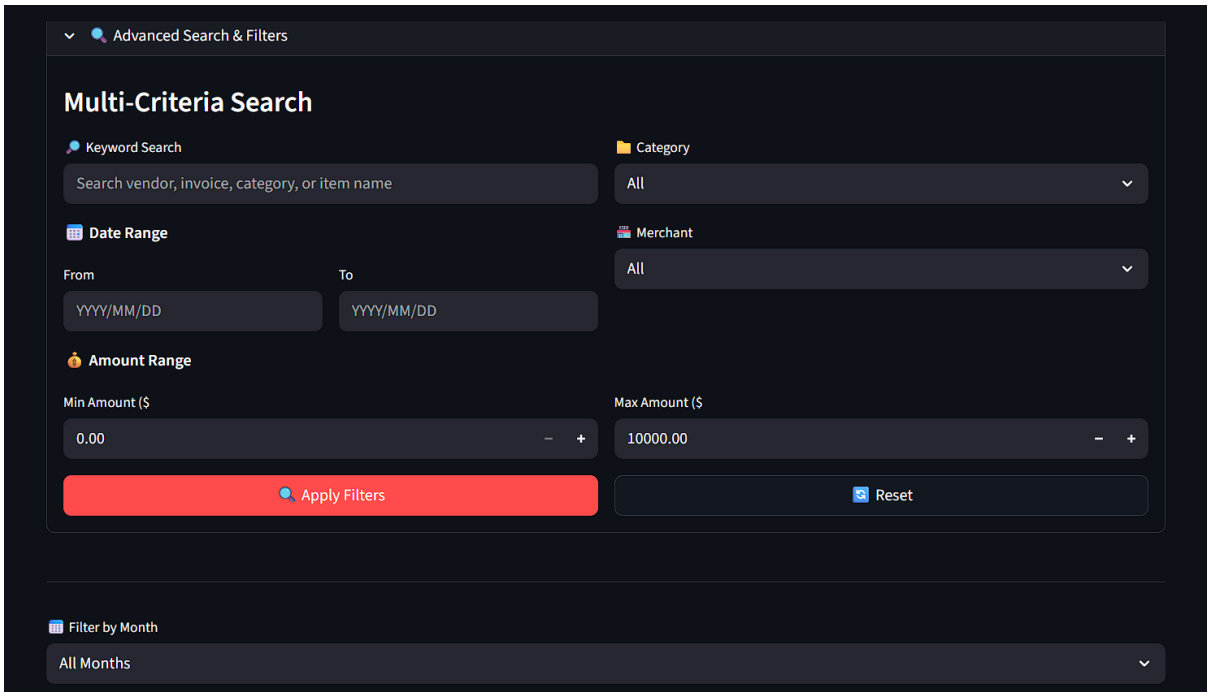




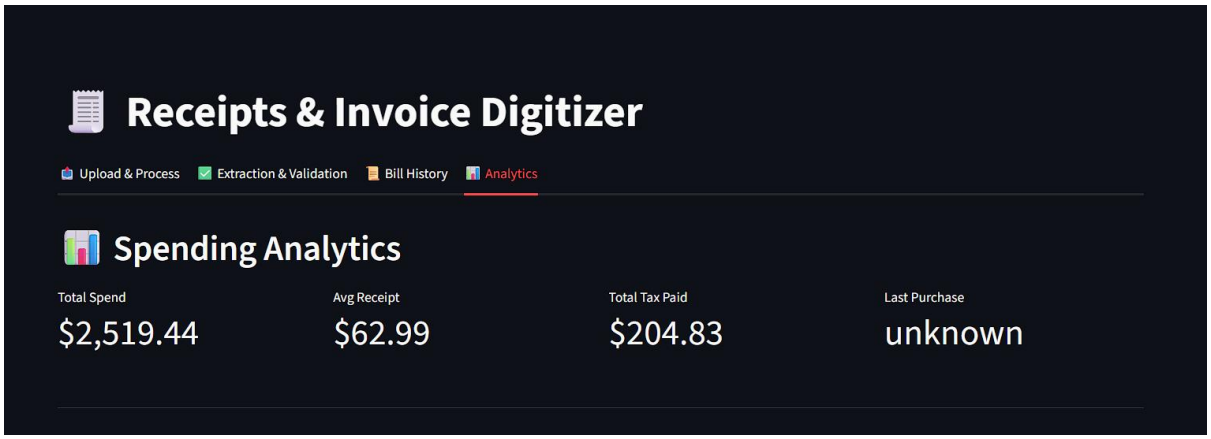
3. Comprehensive Invoice Management & Analysis

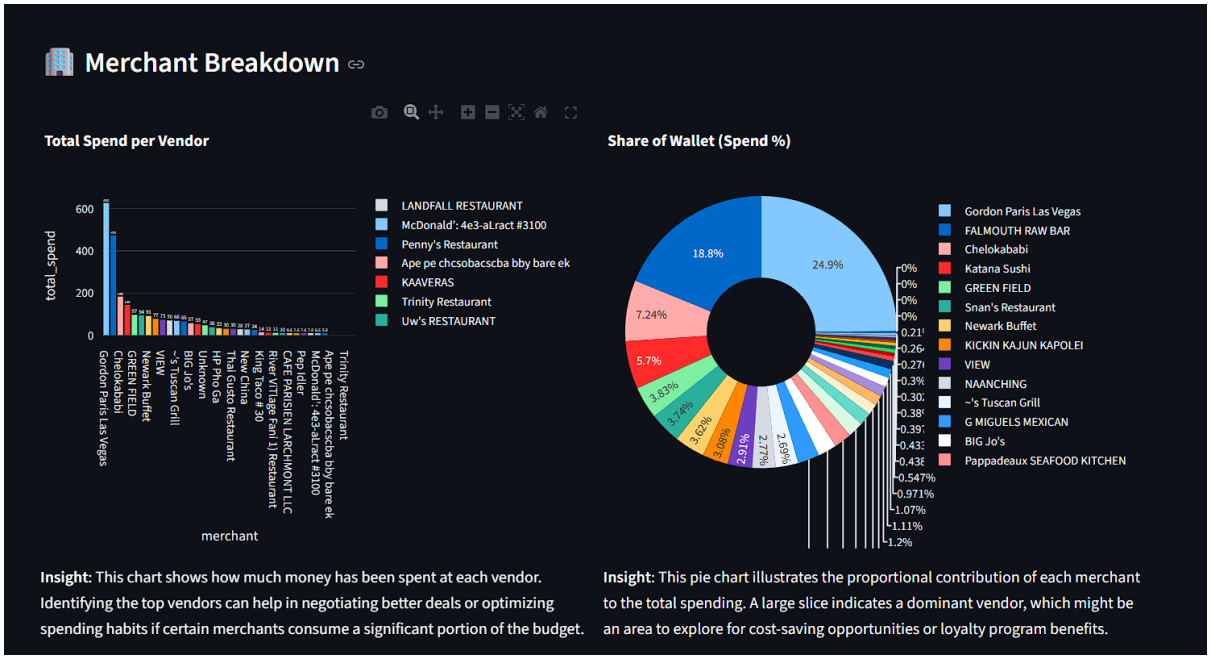


4. Advance Search And Filter



5. Analytics





13. Error Handling & Validation

The system employs a comprehensive, multi-layered error handling and validation framework designed to maintain strict data integrity and operational resilience across the entire digitization lifecycle. Beginning at the ingestion point, the system rigorously validates incoming inputs for file corruption, format compliance, and security risks to prevent system instability. During the processing phase, structured exception handling manages runtime anomalies—such as OCR low-confidence scores or network timeouts—by utilizing automatic retry mechanisms and isolating problematic files into exception queues rather than halting the workflow.

10.1 Upload-Level Validation

Upload-level validation acts as the critical gatekeeper for the system, preventing invalid or malicious data from entering the digitization pipeline at the source. This mechanism strictly enforces file format compatibility by accepting only JPG, PNG, and PDF documents, while simultaneously applying maximum file size limits to ensure optimal performance and resource allocation. To prevent redundancy, the system utilizes file hashing algorithms to instantly detect and reject unchanged re-uploads, ensuring that duplicate files do not consume unnecessary processing power.

10.2 Ingestion & Preprocessing Safety

The Ingestion and Preprocessing Safety layer serves as a robust buffer that ensures continuous system operation even when encountering imperfect input or resource constraints. It handles corrupted or partially readable files gracefully by isolating these exceptions and logging specific errors rather than allowing them to crash the application or stall the queue. To protect system memory and prevent performance bottlenecks, the module enforces strict page limits on PDF documents, effectively mitigating the risk of resource exhaustion during the processing of large files.

10.3 OCR & AI Extraction Handling

To ensure structural consistency and seamless integration, the system strictly enforces a JSON-only output format from the Gemini AI, guaranteeing that all extracted data conforms to a standardized, machine-readable schema. It actively monitors these responses to instantly detect and flag malformed syntax or incomplete data that falls short of validation requirements. In instances where the AI generation fails or produces unusable output, the system automatically triggers a fallback mechanism that retrieves the raw OCR text, enabling a secondary extraction attempt to preserve data availability and accuracy despite potential AI anomalies.

10.4 Regex & NLP Fallback Recovery

To maximize extraction accuracy, the system employs a hybrid refinement strategy that first scans the primary AI output to identify specific fields that are missing or flagged as low-confidence. It then attempts to recover these "weak" data points using targeted regex-based extraction patterns for structured elements like dates, transaction totals, tax amounts, and currency codes, while simultaneously utilizing the spaCy NLP model to contextually detect vendor names. Crucially, this secondary process operates non-destructively; it selectively overrides only the identified weak fields, strictly preserving the high-quality results where the AI model has demonstrated strong confidence.

10.5 Data Normalization Controls

The Data Normalization Controls serve as the final standardization layer, systematically converting all extracted values into safe, consistent formats to ensure uniformity across the dataset. This process includes standardizing text fields to uppercase for improved consistency

and searchability, while simultaneously applying predefined default values to automatically fill any missing or null fields.

10.6 Amount Validation

- Supports both tax-inclusive and tax-exclusive bills
- Allows tolerance for OCR rounding errors
- Displays warnings instead of blocking user actions

10.7 Duplicate Detection

- Detects duplicates using invoice number, vendor, date, and amount
- Prevents accidental double storage of bills

10.8 Database Safety

- Uses transaction-based inserts
- Rolls back on failures
- Maintains referential integrity with foreign keys

10.9 User Feedback

- Clear warnings and error messages
- No application crashes on failure
- User remains in control of final actions

14. Project Work Timeline

Day	Date	Work Description
Day 1	29 Dec 2025	Project planning, requirement analysis, and finalization of system architecture.
Day 2	30 Dec 2025	Database schema design with tables, foreign keys, and referential integrity setup.
Day 3	31 Dec 2025	Implementation of basic project structure and folder organization.
Day 4	01 Jan 2026	Development of upload module with file format validation (JPG, PNG, PDF).
Day 5	02 Jan 2026	Implementation of file size limits and file hashing for duplicate upload prevention.

Day 6	03 Jan 2026	Integration of ingestion safety checks and corrupted file handling.
Day 7	04 Jan 2026	PDF preprocessing with page limit enforcement to prevent memory overload.
Day 8	05 Jan 2026	OCR integration using Tesseract and confidence score monitoring.
Day 9	06 Jan 2026	Implementation of structured exception handling for OCR failures.
Day 10	07 Jan 2026	Integration of AI for structured JSON-based data extraction.
Day 11	08 Jan 2026	JSON schema validation and malformed response detection.
Day 12	09 Jan 2026	Fallback mechanism implementation using raw OCR text.
Day 13	10 Jan 2026	Development of regex-based extraction for dates, totals, tax, and currency fields.
Day 14	11 Jan 2026	Integration of NLP model for vendor name detection.
Day 15	12 Jan 2026	Hybrid refinement logic to override only low-confidence fields.
Day 16	13 Jan 2026	Data normalization module (uppercase conversion and default values).
Day 17	14 Jan 2026	Amount validation logic with tax-inclusive and tax-exclusive support.
Day 18	15 Jan 2026	Implementation of OCR rounding tolerance and warning system.
Day 19	16 Jan 2026	Duplicate detection using invoice number, vendor, date, and amount.
Day 20	17 Jan 2026	Transaction-based database insert implementation.
Day 21	18 Jan 2026	Rollback mechanism for database failure handling.
Day 22	19 Jan 2026	User feedback system with clear warnings and non-blocking errors.
Day 23	20 Jan 2026	Dashboard development with search and filter functionality.
Day 24	21 Jan 2026	Optimization of database queries and report generation.

Day 25	22 Jan 2026	Integration testing of complete digitization pipeline.
Day 26	23 Jan 2026	Exception queue handling and retry mechanism testing.
Day 27	24 Jan 2026	Performance testing with large PDF files.
Day 28	25 Jan 2026	Security validation and edge case testing.
Day 29	26 Jan 2026	Bug fixing and refinement of validation layers.
Day 30	27 Jan 2026	UI improvement and user experience optimization.
Day 31	28 Jan 2026	End-to-end system testing with real sample invoices.
Day 32	29 Jan 2026	Documentation drafting for validation and error handling framework.
Day 33	30 Jan 2026	Documentation drafting for OCR and AI extraction handling.
Day 34	31 Jan 2026	Documentation drafting for fallback and normalization modules.
Day 35	01 Feb 2026	Preparation of system architecture diagrams and workflow charts.
Day 36	02 Feb 2026	Preparation of project presentation slides.
Day 37	03 Feb 2026	Internal review and corrections based on feedback.
Day 38	04 Feb 2026	Final optimization and cleanup of source code.
Day 39	05 Feb 2026	Final database consistency checks and validation testing.
Day 40	06 Feb 2026	Final testing of duplicate detection and amount validation.
Day 41	07 Feb 2026	Preparation of user manual and usage guidelines.
Day 42	08 Feb 2026	Backup creation and deployment preparation.
Day 43	09 Feb 2026	Final integration testing and performance benchmarking.
Day 44	10 Feb 2026	Error handling stress testing and failure simulation.
Day 45	11 Feb 2026	Final UI/UX polishing and layout adjustments.
Day 46	12 Feb 2026	Proofreading complete documentation.
Day 47	13 Feb 2026	Final project review and validation.

Day 48	14 Feb 2026	Minor corrections and final refinements.
Day 49	15 Feb 2026	Final packaging and submission preparation.
Day 50	16 Feb 2026	Project submission and final presentation/demo.

15. Conclusion

The Receipt and Invoice Digitization System was developed in a step-by-step manner to convert manual receipt handling into an automated and reliable process. First, the system architecture and database were designed to ensure proper data storage and smooth workflow. Secure file upload features were added with format validation, size limits, and duplicate detection to prevent invalid data from entering the system.

Next, OCR was integrated to extract text from images and PDFs. AI-based structured extraction was implemented to convert the raw text into organized JSON data. To improve accuracy, validation checks, fallback mechanisms, and regex/NLP-based refinement were added to handle missing or low-confidence fields. Data normalization ensured consistent formatting across all records.

Finally, strong validation rules, duplicate detection logic, transaction-based database operations, and clear user feedback mechanisms were implemented. Search and filter options were added to enhance usability.

Overall, the project successfully delivers a secure, accurate, and efficient system for digitizing receipts and invoices, reducing manual effort while maintaining data integrity and reliability.

16. References

[1] Python Software Foundation, *Python Documentation*, 2025. [Online]. Available: https://docs.python.org/3/

[2] Streamlit Inc., *Streamlit Documentation*, 2025. [Online]. Available: https://docs.streamlit.io/

[3] SQLite Consortium, *SQLite Documentation*, 2025. [Online]. Available: https://www.sqlite.org/docs.html

[4] Tesseract OCR, *Tesseract OCR Documentation*, 2025. [Online]. Available: https://tesseract-ocr.github.io/

[5] pdf2image, *pdf2image Library Documentation*, 2025. [Online]. Available: https://pypi.org/project/pdf2image/

[6] spaCy, *spaCy Usage Documentation*, 2025. [Online]. Available: https://spacy.io/usage

[7] Plotly Technologies Inc., *Plotly Python Documentation*, 2025. [Online]. Available: https://plotly.com/python/

[8] Pandas Development Team, *Pandas Documentation*, 2025. [Online]. Available: https://pandas.pydata.org/docs/

[9]] Groq, *Groq API Documentation*, 2025. [Online]. Available: https://console.groq.com/docs

[10] Speech and Language Processing, D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Draft, 2023. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/